

Plateforme Mastering Live Debix

Spécification Phase 1a — V4

Console numérique 8 canaux end-to-end
Intègre l'investigation critic V3 (6 workers, 622s)

Document de référence technique

2026-04-22 — rev V4

Table des matières

1	Ce qui change depuis V3	2
1.1	Synthèse des findings	2
1.2	Ce qu'on garde de V3	2
1.3	Ce qui est corrigé	2
2	Architecture cible V4 (identique V3)	3
2.1	Les 4 PCMs ALSA (+ gadget USB)	3
3	Root cause V2.7 -110 timeout	3
3.1	Bug primaire : widget fantôme PIPELINE_DEMUX_3	3
3.2	Bug secondaire : guard is_same_sched IPC3	3
3.3	Leçon V4	4
4	Bugs dans le squelette V3 (corrigés en V4)	4
5	Phasage V4	4
6	Phase 1a.1 MVP — squelette m4 corrigé	5
6.1	Architecture	5
6.2	Hypothèses de design	5
6.3	sof-imx8mp-tac5212-masterV4.m4 (top-level)	5
6.4	sof-imx8mp-tac5212-pipe-effects-capture.m4	6
6.5	sof-imx8mp-tac5212-pipe-effects-playback.m4	7
6.6	Board config Kconfig (sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf)	8
6.7	Tests Phase 1a.1 MVP	8
7	Phase 1a.2 — Tap HP_Monitor post-effets OUT	8
8	Phase 1a.3 — Mixer (blocker confirmé)	9
9	Questions pour l'investigation V4	9
10	Livrables Phase 1a.1 MVP V4	9
11	Protocole	10

1 Ce qui change depuis V3

Les six workers de l'investigation critic du 2026-04-22 (gemini-3-pro, kimi-k2.5, glm-5.1, claude-code, minimax, qwen) ont analysé le squelette V3 et le code SOF upstream. Consensus : **V3 est viable mais le squelette m4 fourni contient 4 bugs qui empêcheraient la compile/charge**. La V4 corrige les bugs et affine le plan.

1.1 Synthèse des findings

Sujet	Verdict	Source
8ch linked supporté par mbdrp/pga/drc	OUI unanime	PLATFORM_MAX_CHANNELS=8, boucles for ch=0; ch<nch dans les 3 composants
pipe-drc-enabled-capture.m4 extensible 8ch	OUI unanime	PCM_CAPABILITIES(..., 2, PIPELINE_CHANNELS) accepte 8ch
pipe-volume-playback.m4 extensible 8ch	OUI unanime	même pattern inversé
Budget CPU HiFi4 8ch @ 1ms	~2-3%; période 1ms tenable	gemini/kimi/minimax estimation cycles; glm conseille 2ms pour marge
Root cause V2.7 -110	2 bugs cumulés identifiés	détail §3
Squelette V3 tel quel compile?	NON — 4 bugs	détail §4
Mixer legacy IPC3 pour 3+ sources	NON	MIXER_MAX_SOURCES=2 hardcodé (mixer.h:33)

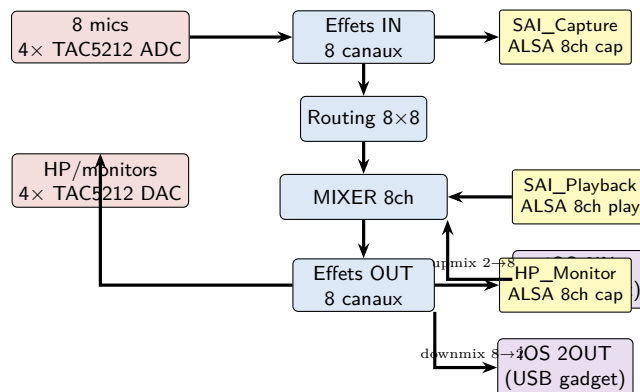
1.2 Ce qu'on garde de V3

- Architecture console 8ch end-to-end (pas de flux 2ch séparé)
- Les 4 PCMs ALSA (SAI_Capture, SAI_Playback, HP_Monitor) + USB gadget iOS
- Phasage : 1a.1 MVP → 1a.4 → Phase 2 → Phase 3
- MVP = 2 pipelines autonomes 8ch, pas de cross-pipeline, pas de tap

1.3 Ce qui est corrigé

1. Skeleton m4 capture/playback : W_PIPELINE obligatoire, DEF_PGA_CONF via vendor tuples, N_DAI_IN retiré de P_GRAPH.
2. Root cause V2.7 -110 : deux bugs cumulés (PIPELINE_DEMUX_3 fantôme + guard is_same_sched).
3. Kconfig : vérifier qu'on compile contre sof/ (déjà complet) et pas sof_fork/ (incomplet).
4. Blocker mixer IPC3 ≤ 2 sources : stratégie pour Phase 1a.2.

2 Architecture cible V4 (identique V3)



2.1 Les 4 PCMs ALSA (+ gadget USB)

PCM	Direction	Ch	Rôle
SAI_Capture	capture	8	Mics post-effets IN (pour enregistrement DAW)
SAI_Playback	playback	8	Retour DAW ASIO, entre dans le mixer (Phase 1a.3)
HP_Monitor	capture	8	Sortie finale post-effets OUT (pour NPU) – Phase 1a.2
USB gadget iOS	2IN/2OUT	2+2	Séparé, connecte au mixer + reçoit downmix – Phase 1a.4

3 Root cause V2.7 -110 timeout

L'investigation a identifié deux bugs cumulés, en cascade :

3.1 Bug primaire : widget fantôme PIPELINE_DEMUX_3

Dans `sof-imx8mp-tac5212-masterlite.m4:88` (V2.7), on a :

```
SectionGraph."master-chain-tap-and-merge" {
    lines [
        dapm(PIPELINE_SINK_5, PIPELINE_DEMUX_3) # <--- PIPELINE_DEMUX_3 JAMAIS DEFINI
        dapm(PIPELINE_SINK_6, PIPELINE_DEMUX_3)
    ]
}
```

Or dans `pipe-masterchain.m4` lignes 71-73, les seuls exports sont :

```
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(1))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), Source Play PCM_ID)
# PAS DE PIPELINE_DEMUX_3 !
```

`m4` expand en chaîne vide. La ligne DAPM devient malformée : pipelines 5 et 6 sont orphelines. Au trigger, le DAI parent attend des données qui ne viennent jamais → -110.

3.2 Bug secondaire : guard `is_same_sched` IPC3

Même si `PIPELINE_DEMUX_3` avait été défini, un deuxième blocage existe dans `sof/src/audio/pipeline/pipel`

```
if (!is_single_ppl && (!is_same_sched || IS_ENABLED(CONFIG_IPC_MAJOR_4))) {
    return 0; // trigger ne propage pas cross-pipeline
}
```

La V2.7 mixait `SCHEDULE_TIME_DOMAIN_TIMER` (PIPE 3, PIPE 5) avec `SCHEDULE_TIME_DOMAIN_DMA` (PIPE 1, PIPE 6). Les `sched_comp` étaient différents → guard actif → pas de propagation du trigger.

3.3 Leçon V4

- Ne jamais référencer un export qui n'existe pas dans le m4 top-level — il n'y a pas de check strict, c'est une expansion silencieuse.
- En IPC3, tous les pipelines connectés par DAPM cross-pipeline doivent partager le même sched_comp (même W_PIPELINE parent). Plus simple : pas de cross-pipeline en MVP.

4 Bugs dans le squelette V3 (corrigés en V4)

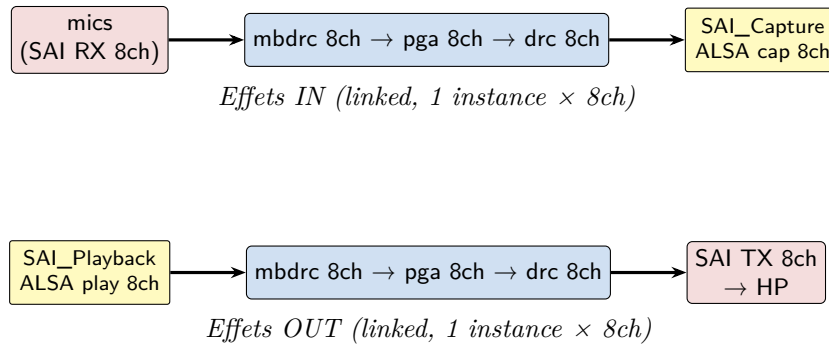
#	Sévérité	Correction V4
B1	CRITICAL	W_PIPELINE(N_PCMC, ...) manquant dans pipe-effects-capture.m4 — ajouté. Idem playback avec N_PCMP.
B2	CRITICAL	DEF_PGA_CONF non défini : V4 intègre le bloc W_VENDORTUPLES + W_DATA repris de pipe-volume-capture.m4.
B3	HIGH	dapm(N_BUFFER(0), N_DAI_IN) retiré du P_GRAPH : DAI_ADD fait le lien via PIPELINE_SINK_N.
B4	HIGH	Ordre DAPM capture inversé (capture = flux DAI → Host, dapm = sink ← source, donc dapm(N_PCMC, N_BUFFER(3)) en bas de chaîne).
B5	MEDIUM	CONFIG_COMP_MIXER=y ajouté au board config pour Phase 1a.3+.

5 Phasage V4

Phase	Livrables	Objectif
1a.1 MVP	2 pipelines 8ch autonomes, effets linked, pas de cross-pipeline	Prouver le pass-through + effets bit-perfect, ouverture/trigger sans -110.
1a.2	Tap HP_Monitor post-effets OUT	Investiguer pattern sûr (voir §7)
1a.3	Injection SAI_Playback via mixer + routing 8×8	Attention blocker mixer ≤ 2 sources
1a.4	USB gadget iOS 2IN/2OUT	Hors DSP pour la partie USB, mais intégration côté mixer
2	USB ASIO gadget 8IN/8OUT	DAW sur host externe
3	Passage linked → per-channel	8 instances d'effets parallèles, CPU permettant

6 Phase 1a.1 MVP — squelette m4 corrigé

6.1 Architecture



6.2 Hypothèses de design

- **Period** 1ms (48 frames) : investigation donne ~2-3% CPU HiFi4, très confortable. Si le moindre xrun, on passe à 2ms.
- **Bit-perfect bypass** : config `NULL → drc_default_pass = audio_stream_copy (drc_generic.c:473)`. Idem mbdrc et pga avec volume Q8.16 = 0x10000.
- **Tous SCHEDULE_TIME_DOMAIN_DMA** par défaut (géré par DAI_ADD).
- **Un seul W_PIPELINE** par `pipeline_id`, piloté par `N_PCMC/N_PCMP`.

6.3 sof-imx8mp-tac5212-masterV4.m4 (top-level)

```
#
# Topology V4 Phase 1a.1 MVP : console 8ch end-to-end, linked effects.
# 2 pipelines 8ch autonomes, SCHEDULE_TIME_DOMAIN_DMA partout.
#
include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# PIPE 1 : SAI7 RX 8ch -> mbdrc -> pga -> drc -> SAI_Capture
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-capture.m4,
    1, 0, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000)

# PIPE 6 : SAI_Playback -> mbdrc -> pga -> drc -> SAI7 TX 8ch
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-playback.m4,
    6, 1, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000)

# DAI_ADD : 7e arg = DAI_PERIODS (2, cohérent baseline drc8ms)
DAI_ADD(sof/pipe-dai-capture.m4,
    1, SAI, 7, tac5212-hifi,
    PIPELINE_SINK_1, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

DAI_ADD(sof/pipe-dai-playback.m4,
    6, SAI, 7, tac5212-hifi,
    PIPELINE_SOURCE_6, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

PCM_CAPTURE_ADD(SAI_Capture, 0, PIPELINE_PCM_1)
PCM_PLAYBACK_ADD(SAI_Playback, 1, PIPELINE_PCM_6)
```

```

DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
  SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
    SAI_CLOCK(bclk, 12288000, codec_consumer),
    SAI_CLOCK(fsyc, 48000, codec_consumer),
    SAI_TDM(8, 32, 255, 255),
    SAI_CONFIG_DATA(SAI, 7, 0)))

```

6.4 sof-imx8mp-tac5212-pipe-effects-capture.m4

```

# Capture 8ch : DAI SAI RX -> B3 -> MBDRc -> B2 -> PGA -> B1 -> DRC -> B0 -> HOST
# Pattern = pipe-drc-enabled-capture.m4 étendu avec mbdrc + pga avant drc.

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')

#
# Host PCM endpoint
#
W_PCM_CAPTURE(PCM_ID, SAI Capture, 0, 2, SCHEDULE_CORE)

#
# Volume (PGA) vendor tuples : repris tel quel de pipe-volume-capture.m4
#
define('CAPTURE_PGA_NAME', Master Capture Volume)
define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))
W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
  LIST(' ', 'SOF_TKN_VOLUME_RAMP_STEP_TYPE "0"',
    'SOF_TKN_VOLUME_RAMP_STEP_MS "250"'))
W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

#
# Controls : volume kmixer + mbdrc/drc bytes
#
define(DEF_PGA_VOLUME, concat('pga_volume_', PIPELINE_ID))
C_CONTROLMIXER(CAPTURE_PGA_NAME, PIPELINE_ID,
  CONTROLMIXER_OPS(volsw, 256 binds the mixer control to volume get/put handlers, 256, 256),
  CONTROLMIXER_MAX(, 32),
  false,
  CONTROLMIXER_TLV(TLV_01, vtlv_m64s2),
  Channel register and shift for Front Left/Right,
  LIST(' ', KCONTROL_CHANNEL(FL, 1, 0), KCONTROL_CHANNEL(FR, 1, 1)))

# MBDRc bytes control blob
include('multiband_drc_coef_default.m4')
define(MBDRc_priv, concat('mbdrc_priv_', PIPELINE_ID))
define(MY_MBDRc_CTRL, concat('mbdrc_ctrl_', PIPELINE_ID))
C_CONTROLBYTES(MY_MBDRc_CTRL, PIPELINE_ID,
  CONTROLBYTES_OPS(bytes, 258 binds the control to bytes get/put handlers, 258, 258),
  CONTROLBYTES_EXTOPS(258 binds the control to bytes get/put handlers, 258, 258),
  , , ,
  CONTROLBYTES_MAX(, SOF_MULTIBAND_DRC_MAX_BLOB_SIZE),
  ,
  MBDRc_priv)

# DRC bytes control blob
include('drc_coef_default.m4')
define(DRC_priv, concat('drc_priv_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_ctrl_', PIPELINE_ID))
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
  CONTROLBYTES_OPS(bytes, 258 binds the control to bytes get/put handlers, 258, 258),
  CONTROLBYTES_EXTOPS(258 binds the control to bytes get/put handlers, 258, 258),
  , , ,
  CONTROLBYTES_MAX(, SOF_DRC_MAX_BLOB_SIZE),
  ,
  DRC_priv)

```

```

#
# Widgets : MBDRC + PGA + DRC
#
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MY_MBDRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, 2, 2, DEF_PGA_CONF, SCHEDULE_CORE,
    LIST(' ', "DEF_PGA_VOLUME"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MY_DRC_CTRL"))

#
# Buffers : B0 host side, B3 DAI side, B1/B2 inter-widgets
#
W_BUFFER(0, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(DAI_PERIODS,
    COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_DAI_MEM_CAP)

#
# Scheduler widget OBLIGATOIRE (bug V2.7 racine secondaire)
#
W_PIPELINE(N_PCMC(PCM_ID), SCHEDULE_PERIOD, SCHEDULE_PRIORITY,
    SCHEDULE_CORE, SCHEDULE_TIME_DOMAIN, pipe_media_schedule_plat)

#
# Graph DAPM : capture = flux DAI -> HOST, dapm(sink, source)
#
P_GRAPH(pipe-effects-capture, PIPELINE_ID,
    LIST(' ',
        'dapm(N_PCMC(PCM_ID), N_BUFFER(0))',
        'dapm(N_BUFFER(0), N_DRC(0))',
        'dapm(N_DRC(0), N_BUFFER(1))',
        'dapm(N_BUFFER(1), N_PGA(0))',
        'dapm(N_PGA(0), N_BUFFER(2))',
        'dapm(N_BUFFER(2), N_MULTIBAND_DRC(0))',
        'dapm(N_MULTIBAND_DRC(0), N_BUFFER(3))'))

#
# Exports pour DAI_ADD (PIPELINE_SINK_N = DAI side buffer)
#
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI_Capture_PCM_ID)

#
# PCM capabilities : s32le 8ch @ 48k
#
PCM_CAPABILITIES(SAI_Capture_PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
    192, 16384, 65536, 65536)

undefine('DEF_PGA_TOKENS') undefine('DEF_PGA_CONF')
undefine('MY_MBDRC_CTRL') undefine('MBDRC_priv')
undefine('MY_DRC_CTRL') undefine('DRC_priv')
undefine('DEF_PGA_VOLUME')

```

6.5 sof-imx8mp-tac5212-pipe-effects-playback.m4

Structure identique, inversée :

- W_PCM_PLAYBACK à la place de W_PCM_CAPTURE
- W_PIPELINE(N_PCMC(PCM_ID), ...)

- Flux : HOST → B0 → MBDR → B1 → PGA → B2 → DRC → B3 → DAI TX
- Graph DAPM inversé : `dapm(N_MULTIBAND_DRC, N_BUFFER(0))`, etc., avec `dapm(N_BUFFER(3), N_DRC(0))` en fin
- Export : `PIPELINE_SOURCE_N = N_BUFFER(3)` pour DAI_ADD

6.6 Board config Kconfig (sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf)

```
CONFIG_IMX8M=y
CONFIG_HAVE_AGENT=n
CONFIG_FORMAT_CONVERT_HIFI3=n
CONFIG_KPB_FORCE_COPY_TYPE_NORMAL=n

# Pour Phase 1a.1 MVP
CONFIG_COMP_DRC=y
CONFIG_COMP_VOLUME=y
CONFIG_COMP_CROSSOVER=y
CONFIG_COMP_MULTIBAND_DRC=y
CONFIG_COMP_MUX=y

# Ajouté V4 pour Phase 1a.3 (mixer legacy, limité 2 sources)
CONFIG_COMP_MIXER=y
```

6.7 Tests Phase 1a.1 MVP

1. **T0 Compile** : `m4 ... masterV4.m4 | alsatplg -c - -o masterV4.tplg` sans erreur, aucun widget orphelin.
2. **T0a Widget count** : la `tplg` contient 4 buffers × 2 pipelines + 3 effects × 2 + 2 PCMs + 2 W_PIPELINE + 2 DAI = attendu ~20 widgets.
3. **T1 Load** : kernel charge `masterV4.tplg`, `dmesg` sans -22 ni -110. Carte + 2 PCMs visibles.
4. **T2 Playback** : `aplay -D plughw:softac5212tdm,1 -c 8 -f S32_LE tone8ch.wav` → son audible, pas de xrun.
5. **T3 Capture** : `arecord -D plughw:softac5212tdm,0 -c 8 -f S32_LE -d 3 mics.wav` → 8 canaux enregistrés.
6. **T4 Full-duplex** : T2 + T3 simultanés, stable 60 s minimum.
7. **T5 Bit-perfect** : null test input SAI_Playback vs output SAI_Capture via loopback HW (TAC out → TAC in court cable), tous effets bypass.
8. **T6 Contrôles** : `amixer controls` liste les 6 contrôles (Volume, MBDR, DRC) par direction. `amixer cset` change param, pas de glitch.
9. **T7 Charge CPU** : `cat /sys/kernel/debug/sof/dsp_perf < 30%`.

7 Phase 1a.2 — Tap HP_Monitor post-effets OUT

Le vrai point dur. L'investigation donne 4 options, par ordre de risque :

1. **MUXDEMUX en fin de pipeline playback** (option V2.7) : en V4, on le teste sans cross-pipeline séparé. Le demux reste dans la **même pipeline** que les effets OUT, et le 2e sink (post-demux) est un buffer local qui **sert de source à un W_PCM_CAPTURE** exporté comme PCM indépendant. Question ouverte : un W_PCM_CAPTURE peut-il être sink d'un widget dans une pipeline playback ? Upstream peut-être jamais testé.
2. **Smart-amplifier reverse pattern** : pattern feedback capture → playback qui existe dans `sof-smart-amplifier`. Inverser le flux pour un tap playback → capture. À explorer.
3. **Pipeline capture co-schéduée** : pipeline capture séparée avec `PIPELINE_SCHED_COMP_6` (piggyback sur playback). Cross-pipeline DAPM, mais `same_sched` garanti.
4. **Loopback hardware** : câble TAC OUT → TAC IN, pattern studio pro. Décision empirique si 1-3 échouent.

Phase 1a.2 design détaillé dans V4.1 après validation Phase 1a.1.

8 Phase 1a.3 — Mixer (blocker confirmé)

MIXER_MAX_SOURCES = 2 dans `sof/src/audio/mixer/mixer.h:33` est un hard-limit IPC3.

Pour 3+ sources (mics routés + SAI_Playback + iOS 2IN), trois stratégies :

1. **Cascade 2 mixers** : `mix1(mics, SAI_Playback) → mix2(mix1_out, iOS_upmixed)`. 2 widgets supplémentaires mais pas de patch SOF.
2. **Patch local** : augmenter MIXER_MAX_SOURCES à 4 dans `mixer.h` et `mixer.c`. À valider que rien d'autre ne présuppose 2.
3. **Upmix iOS dans SAI_Playback avant mixer** : le routing 8×8 absorbe les 2 canaux iOS en slot dans le flux 8ch avant le mixer, qui reste à 2 sources (mics vs Playback combiné).

Décision V4 : **option 3** (routing absorbe iOS), la plus propre. Alternative 1 si ça bloque.

9 Questions pour l'investigation V4

V3 a confirmé la faisabilité architecturale. V4 doit valider la concrétisation :

1. **Revue du squelette m4 V4 capture** (section 6.3) : compile-t-il via `alsatplg` ? `W_VENDORTUPLES` pour `DEF_PGA_CONF` correspond-il au pattern `sof/pipe-volume-capture.m4` ligne par ligne ? `SOF_MULTIBAND_DRC` et `SOF_DRC_MAX_BLOB_SIZE` sont-ils définis dans les headers inclus ?
2. **Blob defaults bypass bit-perfect** : `multiband_drc_coef_default.m4` et `drc_coef_default.m4` upstream initialisent-ils les composants en mode **bypass** (`enabled=0`) ou en mode **processing actif** ? Si actif, le test T5 (null test) échouera et il faut patcher les blobs.
3. **Pattern tap playback post-effets (Phase 1a.2)** : dans `sof/tools/topology/topology1/`, y a-t-il un seul exemple où un `W_PCM_CAPTURE` est connecté comme sink d'un widget DANS UNE PIPELINE PLAYBACK ? Si non, est-ce que le framework SOF IPC3 l'autorise (recherche dans `pcm.m4` et `src/ipc/ipc3/handler.c`) ?
4. **Smart-amplifier inverse** : `sof-smart-amplifier.m4` fournit-il un exemple de feedback **play-back** → **capture** qu'on pourrait copier pour `HP_Monitor`, ou le pattern est-il fondamentalement unidirectionnel (feedback capture → playback uniquement) ?
5. **Routing 8×8 absorbant iOS** : le composant `mux` en mode routing (matrix identity avec remapping) peut-il accepter **2 sources** (mics 8ch + iOS upmix 2→8ch) et produire 1 sortie 8ch ? Ou faut-il un mixer pour la sommation ? (cf. `mux.h` `MUX_MAX_STREAMS=4` IPC3).
6. **Squelette pipe-effects-playback.m4** : en inversant le flux capture, quels pièges (buffer capabilities `PLATFORM_HOST_MEM_CAP` vs `PLATFORM_DAI_MEM_CAP`, ordre DAPM, export `PIPELINE_SOURCE`) différent de la capture ?
7. **Latence mesurée vs estimée** : l'estimation 2-3% CPU @ 1ms est théorique. Existe-t-il un benchmark SOF sur HiFi4 i.MX8MP avec `mbdrc` 8ch qu'on peut citer ? Si non, quel outil embarqué (`sof-logger`, `debugfs`) permet de mesurer précisément ?

10 Livrables Phase 1a.1 MVP V4

- `sof/tools/topology/topology1/sof-imx8mp-tac5212-masterV4.m4`
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-capture.m4`
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-playback.m4`
- Patch `sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf` : `+CONFIG_COMP_MIXER=y`
- `meta-local/recipes-kernel/linux/files/test-phase1a1-mvp.sh` (tests T0–T7)

11 Protocole

- Aucune modification SOF source sans **critic_analyze** préalable (mode DOUX, fenêtre 2000s post-analyze).
- Un changement = un commit = validation utilisateur.
- Gate **m4** | **alsatplg** OBLIGATOIRE avant tout deploy sur board.
- Si un test Tn échoue, **critic_analyze** AVANT de modifier le code.
- Les fichiers V2.7 buggés (**masterlite.m4**, **pipe-masterchain.m4**, **pipe-mastertap.m4**, **pipe-sai-merge.m4**, **mux_route_sai_merge.m4**, **demux_route_default.m4**) seront **supprimés** en même temps que l'ajout des fichiers V4.